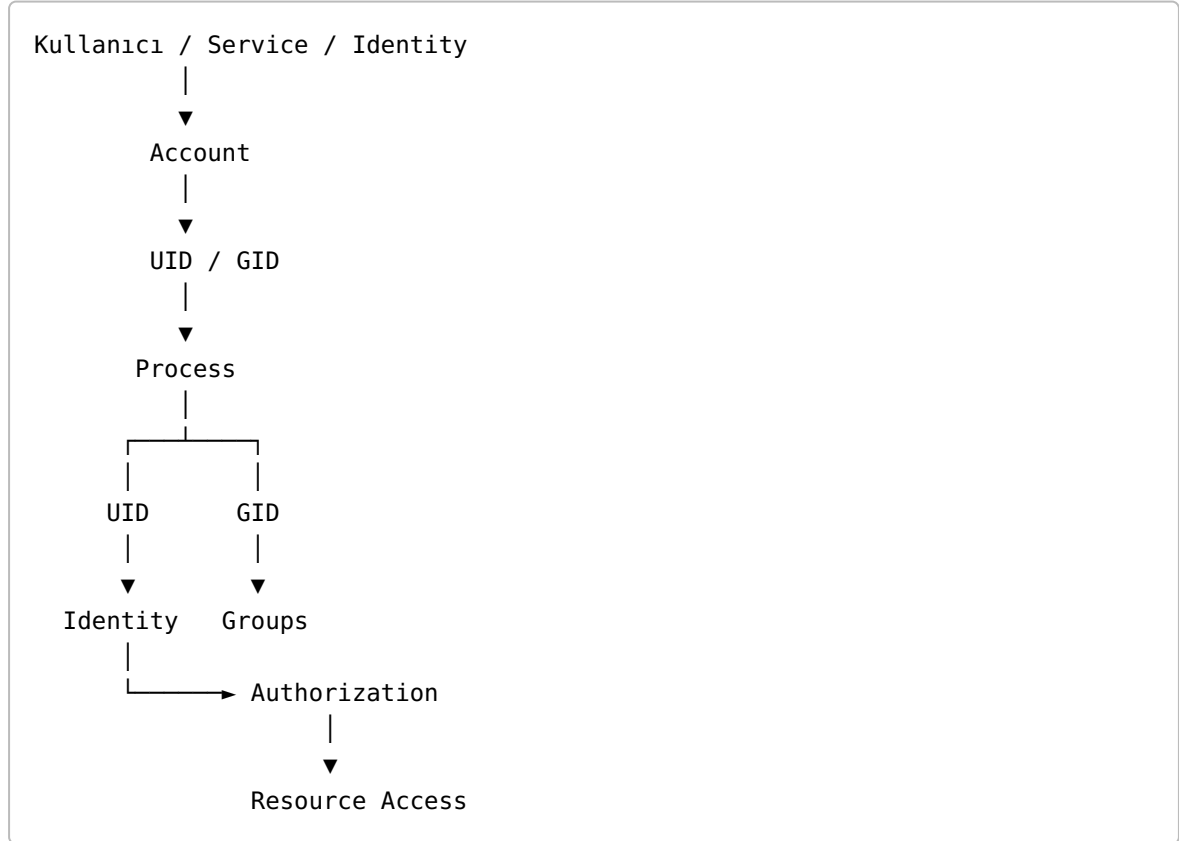


BÖLÜM 1 — Linux'un Kimlik Modelinin Temelleri

Bölüm 0'da genel olarak **identity**, **identifier**, **account**, **authentication**, **authorization**, **privilege** ve **permission** kavramlarını öğrendik.

Artık bu kavramları doğrudan Linux üzerinde incelemeye başlayabiliriz.

Linux'un identity modelini anlamamanın temeli şudur:



Linux'ta bir kimliği yalnızca "kullanıcı adı" olarak düşünmek yeterli değildir.

İşletim sistemi açısından kullanıcı kimliği esas olarak **sayısal kimlikler ve process credentials** üzerinden temsil edilir.

Bu bölümün sonunda şunları anlayabiliyor olmalıyız:

- UID nedir?
- UID 0 neden özeldir?
- Normal kullanıcı ile system/service account arasındaki fark nedir?
- RUID, EUID, SUID ve FSUID nedir?
- Process credential nedir?
- GID nedir?
- Primary group ve supplementary group nedir?
- Linux bir kullanıcının gruplarını nasıl temsil eder?
- `/etc/passwd` ve `/etc/group` ne işe yarar?

- Linux bir process'in kimliğini nasıl belirler?
- Bir process'in UID/GID bilgileri nasıl incelenir?
- "Kullanıcı", "UID", "process" ve "group" neden aynı şey değildir?

1.1 UID Nedir?

UID = User ID

UID (User ID) Linux sisteminde bir kullanıcı kimliğini tanımlamak için kullanılan sayısal identifier'dır.

Örneğin:

```
alice → UID 1000
bob   → UID 1001
```

Linux açısından:

```
Alice
 |
└─ UID = 1000
```

şeklinde bir ilişki vardır.

Ancak burada önemli bir ayrım yapmalıyız:

UID, identity'nin kendisi değildir. UID, identity'yi sistem içinde temsil eden bir identifier'dır.

Yani:

```
Identity
 |
└─ Identifier
    |
    └─ UID = 1000
```

şeklinde düşünmek daha doğrudur.

1.2 UID neden sayısal?

Linux'un çekirdeği için:

```
alice
```

gibi bir kullanıcı adı doğrudan yeterli değildir.

Çekirdek ve sistem bileşenleri için daha temel ve hızlı karşılaştırılabilen değer:

```
1000
```

gibi sayısal UID'dir.

Kullanıcı adı:

```
alice
```

bir insan tarafından okunabilen representation'dır.

UID:

```
1000
```

ise işletim sisteminin kimlik modelindeki sayısal identifier'dır.

Bu yüzden:

```
alice → 1000
```

gibi düşünebiliriz.

Bu eşleştirmeyi sistemin kullanıcı veritabanı üzerinden öğrenebiliriz.

Örneğin:

```
getent passwd alice
```

1.3 UID 0

Linux'taki en önemli UID:

```
UID 0
```

'dır.

Geleneksel Unix/Linux sistemlerinde UID 0, `root` hesabıyla ilişkilendirilir.

```
root
|
└─ UID 0
```

Bu yüzden UID 0, klasik Unix privilege modelinin merkezindedir.

Örneğin:

```
alice → UID 1000
bob   → UID 1001
root  → UID 0
```

Ancak şu ayrım önemlidir:

UID 0 = modern Linux'ta sınırsız ve koşulsuz yetki garantisi değildir.

Modern Linux'ta yetki modeli yalnızca UID'den oluşmaz.

Örneğin:

```
UID
+
Capabilities
+
User Namespace
+
ACL
+
LSM
+
diğer güvenlik mekanizmaları
```

birlikte rol oynayabilir.

Bu nedenle:

```
UID 0
```

çok önemli bir ayrıcalık göstergesidir; fakat Linux'un bütün security modelini tek başına açıklamaz.

1.4 Normal UID ve System UID

Linux sistemlerinde yalnızca insanlar için hesaplar bulunmaz.

Bir uygulama veya servis de kendi identity'sine sahip olabilir.

Örneğin:

```
root
postgres
www-data
daemon
nobody
```

gibi hesaplar görebiliriz.

Bunların amacı genellikle bir servisin veya sistem bileşeninin kendi kullanıcı kimliğiyle çalışmasını sağlamaktır.

Örneğin:

```
PostgreSQL
├── postgres user
│   └── UID
```

Bu yaklaşımın önemli bir güvenlik avantajı vardır:

Her servisi doğrudan root olarak çalıştırmak yerine ayrı bir identity ile çalıştırmak, yetkileri sınırlandırmaya yardımcı olur.

Bu **least privilege** prensibiyle ilişkilidir.

1.5 UID aralıkları

Bir Linux dağıtımında UID'lerin hangi aralıklarda kullanılacağı dağıtıma ve sistem politikasına bağlı olabilir.

Genel olarak:

```
UID 0
├── root
```

```
normal kullanıcı UID'leri
└─ insan kullanıcılar için kullanılan aralıklar

system/service UID'leri
└─ servisler için kullanılan aralıklar
```

şeklinde bir ayırım görebiliriz.

Ancak:

"1000 ve üzeri kesinlikle insan, 1000 altı kesinlikle system account"

gibi evrensel bir Linux kuralı yoktur.

Dağıtım ve yapılandırmaya göre değerler değişebilir.

Bu nedenle gerçek sistemi anlamanın en güvenilir yolu sistemin kendi kayıtlarını incelemektir:

```
getent passwd
```

1.6 Kullanıcı adı ile UID aynı şey değildir

Şu iki kavramı karıştırmamak gerekir:

```
alice
```

ve:

```
1000
```

Bunlar aynı şey değildir.

```
alice
└─ account'taki kullanıcı adı

1000
└─ UID
```

Yani:

Username bir representation'dır; UID sayısal identifier'dır.

Aynı şekilde:

```
UID = 1000
```

gördüğümüzde bunun bize tek başına insanın kim olduğunu anlatmak zorunda olmadığını da unutmamalıyız.

UID'nin hangi kullanıcıya karşılık geldiğini kullanıcı veritabanından öğrenebiliriz.

1.7 Process Identity

Buraya kadar kullanıcıdan bahsettik.

Şimdi daha önemli bir noktaya geliyoruz:

Linux yalnızca kullanıcıları değil, **process'leri de security context ile çalıştırır.**

Bir kullanıcı bir program çalıştırdığında:

```
Alice
UID 1000
  |
  | execute
  ▼
Process
```

process'in kendi credentials'ı oluşur.

Bu credentials içerisinde UID/GID bilgileri bulunur.

Örneğin normal bir process:

```
RUID = 1000
EUID = 1000
SUID = 1000
FSUID = 1000
```

gibi değerlere sahip olabilir.

1.8 Process Credential Nedir?

Process credentials, Linux'un bir process'i güvenlik açısından değerlendirirken kullandığı kimlik ve yetkiyle ilişkili bilgilerin bütünüdür.

Bunlar arasında UID ve GID türleri bulunur.

Basitleştirilmiş model:



Bu bölümde önce UID tarafını, daha sonra GID tarafını ele alacağız.

1.9 RUID — Real User ID

RUID (Real User ID), process'in ilişkili olduğu gerçek kullanıcı kimliğini temsil eden UID'dir.

Örneğin Alice:

UID = 1000

ve Alice bir program çalıştırıyor.

Normal durumda:

RUID = 1000
EUID = 1000

olur.

RUID'yi zihinde şöyle tutabiliriz:

"Bu process hangi kullanıcı bağlamıyla ilişkili?"

Burada "gerçek" kelimesini fiziksel anlamda düşünmemek gerekir.

RUID, process'in security credentials'ındaki belirli bir UID'dir.

1.10 EUID — Effective User ID

EUID (Effective User ID), process'in bazı güvenlik ve erişim kontrollerinde etkin olarak kullanılan UID'dir.

Buradaki anahtar kelime:

Effective = Etkin

dir.

Örneğin:

RUID = 1000
EUID = 1000

olan normal bir process düşünelim.

Bu durumda ikisi aynıdır.

Ama bazı privilege transition durumlarında:

RUID = 1000
EUID = 0

gibi bir durum oluşabilir.

Burada:

RUID
1000
↓
Alice'in kullanıcı bağlamıyla ilişkili

EUID
0
↓
root UID'si etkin

şeklinde bir ayrım vardır.

İşte RUID ile EUID arasındaki asıl fark budur.

1.11 RUID ile EUID neden farklı?

Çünkü şu iki bilgi aynı olmak zorunda değildir:

Process hangi kullanıcı bağlamıyla ilişkili?

ve:

Process şu anda hangi UID ile etkin olarak çalışıyor?

Örneğin:

```
Alice
UID 1000
|
|
▼
Process
RUID = 1000
EUID = 0
```

Burada:

RUID → Alice'in UID'si
EUID → root'un UID'si

olabilir.

Bu mekanizma özellikle:

```
sudo
setuid
setuid executable
privilege transitions
```

gibi konularda karşımıza çıkar.

1.12 Normal Process'te neden fark görmüyoruz?

Çünkü çoğu normal process'te:

RUID = EUID

olur.

Örneğin:

Alice → UID 1000

Terminal

└─ bash

└─ program

ve:

RUID = 1000

EUID = 1000

olabilir.

Bu yüzden günlük Linux kullanımında:

"RUID ile EUID aynı şey."

gibi bir izlenim oluşabilir.

Ama bunlar aynı kavram değildir.

Sadece **değerleri aynı olabilir**.

Bu çok önemli bir ayrımdır.

1.13 SUID — Saved Set-user-ID

SUID (Saved Set-user-ID), process credential'larında privilege transition mekanizmalarıyla ilişkili olarak saklanan UID'dir.

SUID'yi:

"Şu anda kullanılan UID"

olarak düşünmemek gerekir.

EUID:

hangi UID etkin?

sorusuyla daha doğrudan ilişkilidir.

SUID ise:

ayrıcalık değişimleri sırasında hangi UID saklanmış?

sorusuyla ilgilidir.

1.14 "SUID" kelimesinin iki farklı kullanımı

Burada Linux öğrenen herkesin dikkat etmesi gereken bir terminoloji problemi vardır.

"SUID" denildiğinde iki farklı şeyden söz ediliyor olabilir.

Process tarafındaki SUID

Saved Set-user-ID

Dosya tarafındaki setuid biti

Örneğin:

-rwsr-xr-x

Buradaki:

s

setuid permission bitidir.

Yani:

Saved Set-user-ID

ile:

```
setuid file permission bit
```

aynı şey değildir.

Fakat aralarında doğrudan ilişkili bir mekanizma vardır.

1.15 Setuid Executable

Root tarafından sahip olunan ve setuid biti aktif bir executable düşünelim:

```
-rwsr-xr-x root root program
```

Bir kullanıcı:

```
Alice  
UID 1000
```

bu programı çalıştırdığında, klasik setuid semantiği nedeniyle process'in effective UID'si executable'ın sahibi olan root'a dönüşebilir.

Basitleştirilmiş olarak:

```
Alice  
UID 1000  
  |  
  | execute  
  ▼  
setuid-root program  
  |  
  ├── RUID = 1000  
  └── EUID = 0
```

Burada neden RUID ve EUID'nin ayrı tutulduğunu çok net görebiliriz.

```
RUID → process Alice'in kullanıcı bağlamıyla ilişkili  
EUID → process root UID'siyle etkin
```

Bu tür ayrışmalar security açısından çok önemlidir.

1.16 FSUID — Filesystem User ID

FSUID (Filesystem User ID), filesystem permission kontrolleriyle ilişkili UID'dir.

Çoğu normal process'te:

```
FSUID = EUID
```

olur.

Dolayısıyla başlangıç seviyesinde:

```
RUID  
EUID  
SUID  
FSUID
```

değerlerinin aynı olduğunu görmek normaldir.

Örneğin:

```
RUID = 1000  
EUID = 1000  
SUID = 1000  
FSUID = 1000
```

FSUID'yi şimdilik şu şekilde hatırlamak yeterlidir:

FSUID → filesystem erişim kontrolleri için kullanılan UID

FSUID'nin ayrıntılı davranışı ileride filesystem security konularında daha detaylı ele alınabilir.

1.17 UID ailesinin tamamı

Artık dört temel UID'yi yan yana koyabiliriz:

UID	Açılım	Temel rol
RUID	Real User ID	Process'in ilişkili olduğu gerçek kullanıcı UID'si
EUID	Effective User ID	Etkin olarak kullanılan UID
SUID	Saved Set-user-ID	Privilege transition için saklanan UID
FSUID	Filesystem User ID	Filesystem permission kontrolleriyle ilişkili UID

Hafıza yöntemi:

```
R = Real  
E = Effective  
S = Saved  
FS = Filesystem
```

1.18 /proc Üzerinden Process UID'lerini Görmek

Linux process'lerinin runtime bilgilerini:

```
/proc
```

üzerinden inceleyebiliriz.

Önce bir process oluşturalım:

```
sleep 300 &
```

PID'yi bul:

```
pgrep sleep
```

Ardından:

```
cat /proc/$(pgrep sleep | head -n1)/status | grep '^Uid:'
```

Örneğin:

```
Uid:    1000    1000    1000    1000
```

Buradaki sıra:

```
Uid:  
  RUID  
  EUID  
  SUID  
  FSUID
```

şeklindedir.

Yani:

```
Uid: 1000 1000 1000 1000
      |    |    |    |
      |    |    |    +--- FSUID
      |    |    +--- SUID
      |    +--- EUID
      +--- RUID
```

1.19 GID Nedir?

UID kullanıcı kimliğini temsil ediyorsa:

GID

grup kimliğini temsil eder.

GID = Group ID

Örneğin:

```
users → GID 100
developers → GID 1001
docker → GID 999
```

gibi değerler bulunabilir.

Bir kullanıcı yalnızca tek bir kimliğe sahip olmak zorunda değildir; aynı kullanıcı birçok grubun üyesi olabilir.

Bu Linux authorization modelinde çok önemlidir.

1.20 User ve Group ilişkisi

Örneğin:

```
Alice
UID = 1000
```

ve:


```
developers
GID = 1001
```

olsun.

Alice bu grubun üyesiye:

```
Alice
UID 1000
├── Primary Group
├── developers
├── docker
└── başka gruplar
```

gibi bir yapı oluşabilir.

Böylece bir dosyanın erişim hakları yalnızca kullanıcıya değil, grubuna göre de değerlendirilebilir.

1.21 Primary Group

Bir kullanıcının bir **primary group**'u vardır.

Örneğin:

```
Alice
UID = 1000
Primary GID = 1000
```

olabilir.

Bu bilgi kullanıcının account bilgisinin bir parçasıdır.

Linux'taki dosya oluşturma davranışlarında primary group önemli rol oynar.

Örneğin Alice bir dosya oluşturduğunda dosyanın sahibi:

```
Alice
```

olurken grup sahibi de process'in group credentials'ına göre belirlenebilir.

Basit örnek:

```
-rw-r--r-- alice developers report.txt
```

Burada:

```
owner = alice  
group = developers
```

şeklinde bir ilişki vardır.

1.22 Supplementary Groups

Bir kullanıcı yalnızca primary group'a sahip değildir.

Ek gruplara:

```
supplementary groups
```

üyesi olabilir.

Örneğin:

```
Alice  
  
Primary:  
  alice  
  
Supplementary:  
  developers  
  docker  
  audio
```

gibi.

Bu gruplar authorization açısından çok önemlidir.

Örneğin:

```
docker
```

grubuna üyelik bazı sistemlerde Docker daemon ile etkileşim açısından güçlü yetkiler doğurabilir.

Bu nedenle:

Grup üyeliği yalnızca organizasyonel bir etiket değildir; security açısından privilege doğurabilir.

1.23 Kullanıcı ile Grup aynı şey değildir

Şunları birbirine karıştırmamak gerekir:

User
UID
Group
GID

Örneğin:

Alice

bir kullanıcı/account'tır.

1000

onun UID'sidir.

developers

bir gruptur.

1001

bu grubun GID'sidir.

Yani:

Alice
|
└─ UID 1000

developers
|
└─ GID 1001

1.24 Linux Kullanıcı Veritabanı: `/etc/passwd`

Linux sistemlerinde kullanıcı account bilgileri çoğu sistemde `/etc/passwd` üzerinden görülebilir.

Örneğin:

```
alice:x:1000:1000:Alice:/home/alice:/bin/bash
```

Bu kaydı kabaca:

```
alice
|
├── username
├── UID = 1000
├── GID = 1000
├── home = /home/alice
└── shell = /bin/bash
```

şeklinde okuyabiliriz.

Klasik format:

```
username:password-placeholder:UID:GID:GECOS:home:shell
```

şeklindedir.

Ancak modern sistemlerde parola bilgisi genellikle `/etc/shadow` gibi ayrı bir yapıda tutulur.

Dolayısıyla `/etc/passwd` gördüğümüzde:

"Buradaki kullanıcı kaydı bütün authentication bilgisini içeriyor."

diye düşünmemeliyiz.

1.25 `/etc/group`

Gruplar için ise:

```
/etc/group
```

dosyası önemlidir.

Örneğin:

```
developers:x:1001:alice,bob
```

şeklinde bir kayıt düşünelim.

Burada:

```
developers
|
|— GID = 1001
|
|— alice
|— bob
```

ilişkisini görürüz.

Ancak modern Linux sistemlerinde kullanıcı ve grup verileri yalnızca `/etc/passwd` ve `/etc/group` ile sınırlı olmak zorunda değildir.

NSS gibi mekanizmalar üzerinden:

```
LDAP
Active Directory
SSSD
NIS
```

gibi harici identity kaynakları da kullanılabilir.

Bu nedenle gerçek sistemi incelemek için:

```
getent passwd
getent group
```

komutları çoğu zaman doğrudan `/etc/passwd` veya `/etc/group` okumaktan daha doğru bir yöntemdir.

1.26 `id` Komutu

Bir kullanıcının identity ve group bilgilerini görmek için en faydalı komutlardan biri:

```
id
```

dir.

Örneğin:

```
uid=1000(alice) gid=1000(alice) groups=1000(alice),1001(developers),
999(docker)
```

gibi bir çıktı görebiliriz.

Bunu şöyle okuyabiliriz:

```
uid=1000(alice)
  |
  └─ Kullanıcının UID'si

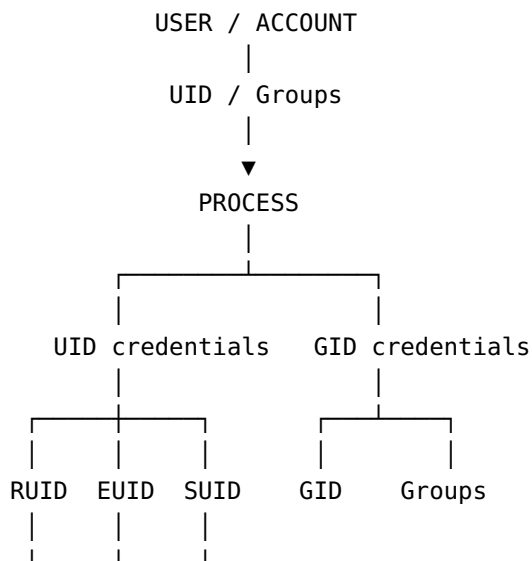
gid=1000(alice)
  |
  └─ Primary GID

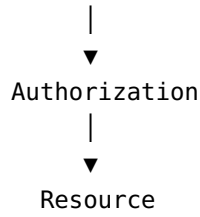
groups=...
  |
  └─ Üyesi olduğu gruplar
```

Bu komut Linux identity modelini anlamak için temel araçlarımızdan biridir.

1.27 Identity → Process → Group ilişkisi

Artık genel yapıyı kurabiliriz:





Bu yapının önemli sonucu şudur:

Kullanıcı, process ve resource aynı security nesnesi değildir.

Bir kullanıcı birden fazla process çalıştırabilir.

Bir process bir kullanıcı adına çalışabilir.

Bir kullanıcı birçok gruba üye olabilir.

Bir process'in etkin UID'si, ilişkili RUID'sinden farklı olabilir.

1.28 Aynı Kullanıcı, Birden Fazla Process

Örneğin Alice:

UID 1000

aynı anda:

```
bash
firefox
python
vim
ssh
```

çalıştırabilir.

Bunların hepsi farklı process'lerdir.

Fakat normal şartlarda aynı kullanıcı credentials'ını paylaşabilirler:

```
Alice
UID 1000
├── bash
├── firefox
└── python
```

```
| vim
| ssh
```

Dolayısıyla:

UID kullanıcıyı temsil eder; PID process'i temsil eder.

Bu ikisini karıştırmamak gerekir.

1.29 UID ve PID farkı

UID = User ID
PID = Process ID

Örneğin:

PID 4217
UID 1000

şunu ifade eder:

4217
→ process'in ID'si

1000
→ process'in ilişkili olduğu kullanıcı UID'si

Yani:

PID ≠ UID

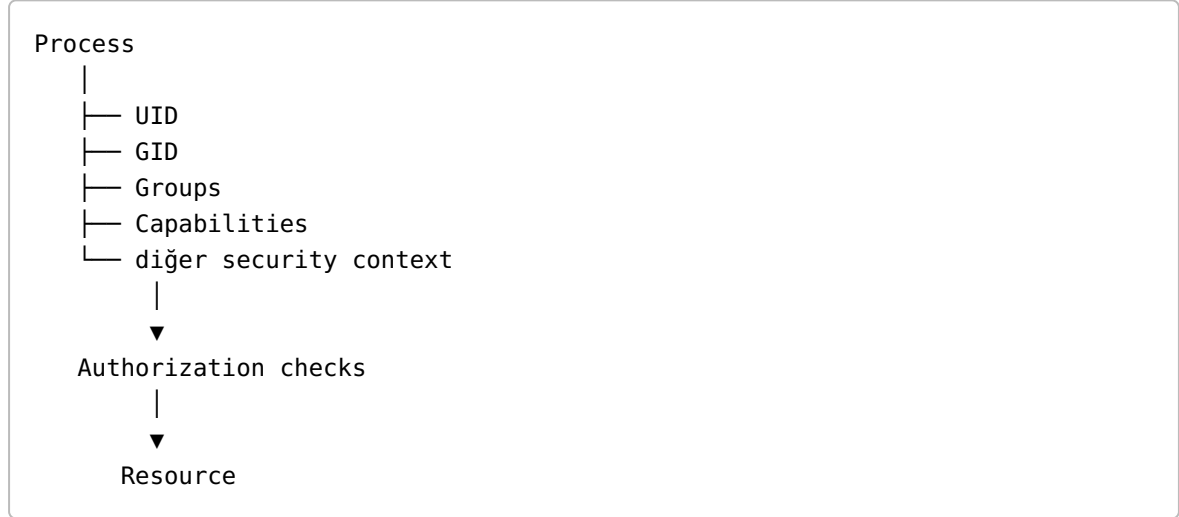
Bir kullanıcı birçok process'e sahip olabilir:

UID 1000
|
├─ PID 4217
├─ PID 4218
├─ PID 4221
└─ PID 4300

1.30 UID ve Permission ilişkisi

Bir process bir resource'a erişmek istediğinde Linux security modelinin çeşitli parçaları devreye girer.

Basitleştirilmiş model:



Örneğin dosya erişiminde:

```
owner
group
others
```

gibi klasik permission modeli rol oynayabilir.

Ama authorization yalnızca:

```
rwX
```

bitlerinden ibaret değildir.

ACL:

```
Access Control Lists
```

gibi mekanizmalar da devreye girebilir.

Daha ileri seviyede:

```
Capabilities
SELinux
```

```
AppArmor  
Namespaces  
seccomp
```

gibi mekanizmalar da security kararlarını etkileyebilir.

1.31 UID 0 ve root kavramını doğru anlamak

Şu cümle fazla basittir:

"root her şeyi yapabilir."

Başlangıç seviyesinde bunun pratik bir açıklama değeri vardır.

Ancak teknik olarak Linux'ta privilege modeli daha geniştir.

Daha doğru model:

```
root  
UID 0  
|  
├── güçlü geleneksel privilege  
├── capabilities  
├── namespace bağlamı  
├── LSM policy  
└── diğer kernel security mekanizmaları
```

Örneğin bir process'in UID 0 olması onu otomatik olarak host sistemin her yerinde sınırsız yetkiye sahip bir process yapmaz.

Özellikle **user namespace** konusu ileride bunu daha net gösterecektir.

1.32 Security açısından neden Process UID'lerine bakıyoruz?

Bir güvenlik sistemi yalnızca:

```
username
```

bilgisine bakmakla yetinmemelidir.

Çünkü saldırı veya privilege transition sırasında önemli olan:

```
hangi kullanıcı?  
hangi process?  
hangi UID?  
hangi effective UID?  
hangi group?  
hangi privilege?  
hangi resource?
```

sorularıdır.

Örneğin:

```
Alice  
UID 1000  
  |  
  ▼  
PID 4217  
  |  
  ├── RUID = 1000  
  ├── EUID = 0  
  └── Groups = ...
```

gibi bir telemetry kaydı çok daha fazla güvenlik bağlamı sağlar.

Bu özellikle EDR ve ITDR için önemlidir.

1.33 ITDR açısından UID

ITDR'de yalnızca:

```
"hangi kullanıcı?"
```

sorusunu sormak yeterli değildir.

Aşağıdaki zincir daha değerlidir:

```
Identity  
  ↓  
Account  
  ↓
```

```
UID
↓
Process
↓
RUID
↓
EUID
↓
Groups
↓
Privilege
↓
Action
↓
Resource
```

Örneğin:

```
Alice
UID 1000
|
▼
ssh
|
▼
PID 4217
|
├─ RUID = 1000
└─ EUID = 1000
|
▼
sudo
|
▼
privilege transition
|
▼
EUID = 0
|
▼
sensitive resource access
```

Burada ITDR açısından sadece son olayı görmek yerine **identity → process → privilege transition → action** zincirini görmek çok daha değerlidir.

1.34 İlk Laboratuvar — Kullanıcı Kimliğini Gör

Önce kendi identity'mizi inceleyelim:

```
id
```

Daha yalnızca UID:

```
id -u
```

Kullanıcı adı:

```
id -un
```

Belirli bir kullanıcı:

```
id root
```

Kullanıcı veritabanı kaydı:

```
getent passwd "$USER"
```

Gruplar:

```
groups
```

ve:

```
id
```

1.35 İkinci Laboratuvar — Process UID'lerini Gör

Bir process oluştur:

```
sleep 300 &
```

PID'yi bul:

```
pgrep sleep
```

Process'i incele:

```
cat /proc/$(pgrep sleep | head -n1)/status | grep '^Uid:'
```

Örneğin:

```
Uid: 1000 1000 1000 1000
```

Bunu:

```
RUID = 1000  
EUID = 1000  
SUID = 1000  
FSUID = 1000
```

olarak oku.

1.36 Üçüncü Laboratuvar — Process ve Kullanıcıyı Birlikte Gör

```
ps -eo pid,ppid,user,uid,euid,cmd
```

Burada process'leri ve kimlik bilgilerini birlikte inceleyebilirsiniz.

Örneğin:

PID	PPID	USER	UID	EUID	CMD
4217	4001	alice	1000	1000	/bin/bash
4230	4217	alice	1000	1000	python app.py

Bu tablo bize:

```
hangi process?  
hangi kullanıcı?  
hangi UID?  
hangi EUID?
```

sorularının cevabını verir.

1.37 Dördüncü Laboratuvar — Group Bilgisi

Kendi kullanıcı bilgini:

```
id
```

ile incele.

Daha sonra:

```
getent group
```

ile sistemdeki grupları incele.

Belirli bir grup:

```
getent group docker
```

gibi sorgulanabilir.

Amaç şunu görmektir:

```
User
↓
UID
↓
Primary GID
↓
Supplementary Groups
↓
Authorization context
```

1.38 En Sık Karıştırılan Kavramlar

UID ≠ Identity

```
Identity
↓
Identifier
```

↓
UID

UID, identity'nin kendisi değildir.

UID ≠ PID

UID → kullanıcı kimliği
PID → process kimliği

User ≠ Process

Bir kullanıcı:

Alice

aynı anda birçok process çalıştırabilir.

RUID ≠ EUID

Aynı değeri taşıyabilirler ama aynı kavram değildir.

RUID = 1000
EUID = 1000

olması ikisini aynı kavram yapmaz.

Permission ≠ Privilege

Permission:

Belirli bir resource üzerinde belirli bir action'a izin

Privilege:

Daha geniş bir yetki / authority

SUID ≠ setuid bit

Process'teki:

Saved Set-user-ID

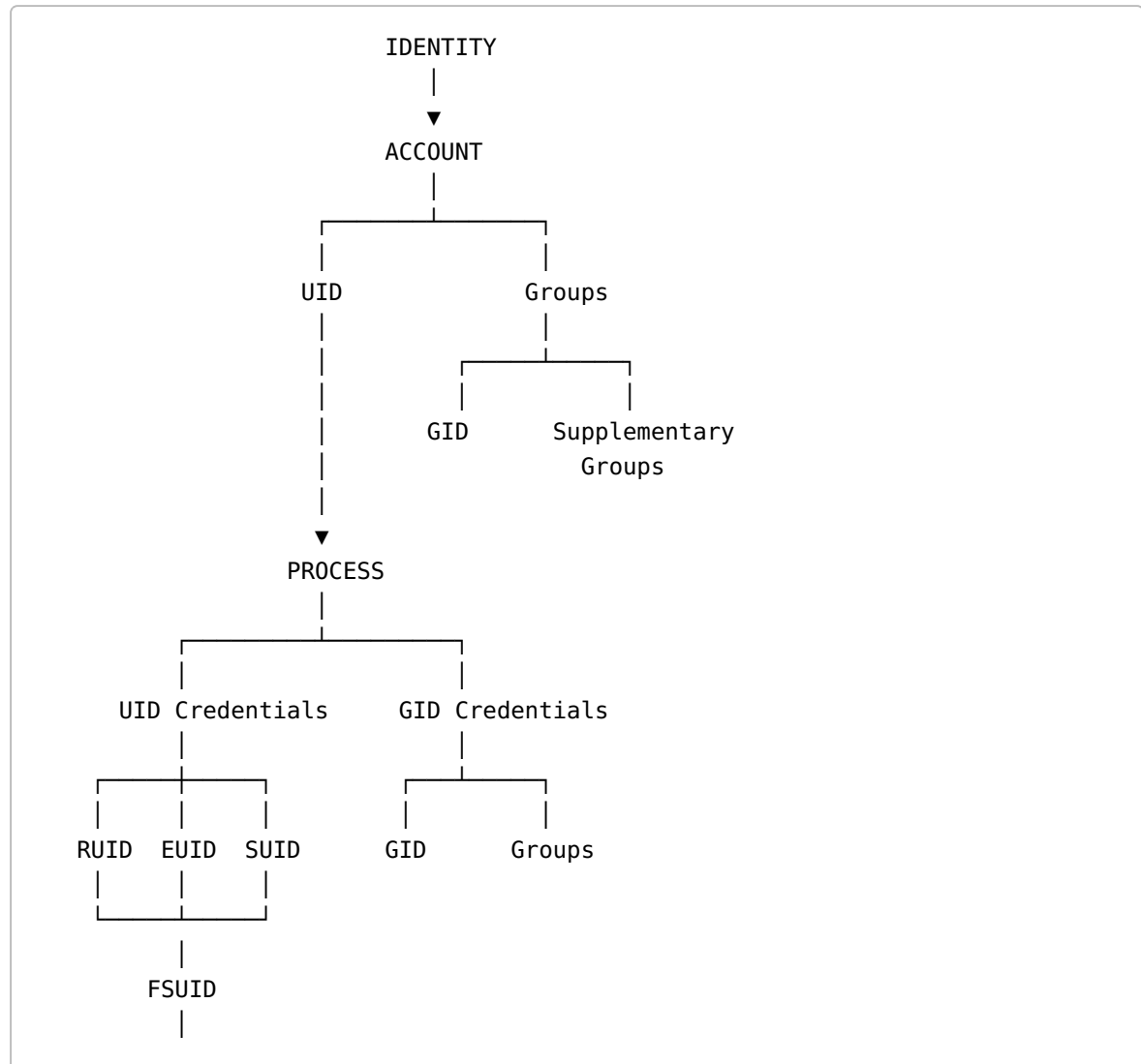
ile executable'daki:

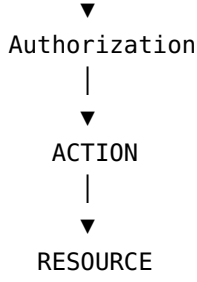
setuid permission bit

aynı kavram değildir.

1.39 Bölümün Büyük Resmi

Artık Linux identity modelini şu şekilde düşünebiliriz:





Bu modelde her parçanın farklı bir görevi vardır.

1.40 Bölümün Özeti

Bu bölümün sonunda şu kavramları birbirinden ayırabiliyor olmalıyız:

Identity

Sistemin tanıdığı güvenlik kimliği.

Account

Identity'nin belirli bir sistemdeki yönetilen temsili.

UID

Kullanıcı identity'sini temsil eden sayısal identifier.

GID

Grup identity'sini temsil eden sayısal identifier.

RUID

Process'in ilişkili olduğu gerçek kullanıcı UID'si.

EUID

Process'in etkin olarak kullandığı UID.

SUID

Ayrıcalık geçişleriyle ilişkili olarak saklanan UID.

FSUID

Filesystem permission kontrolleriyle ilişkili UID.

Primary Group

Kullanıcının temel grubu.

Supplementary Groups

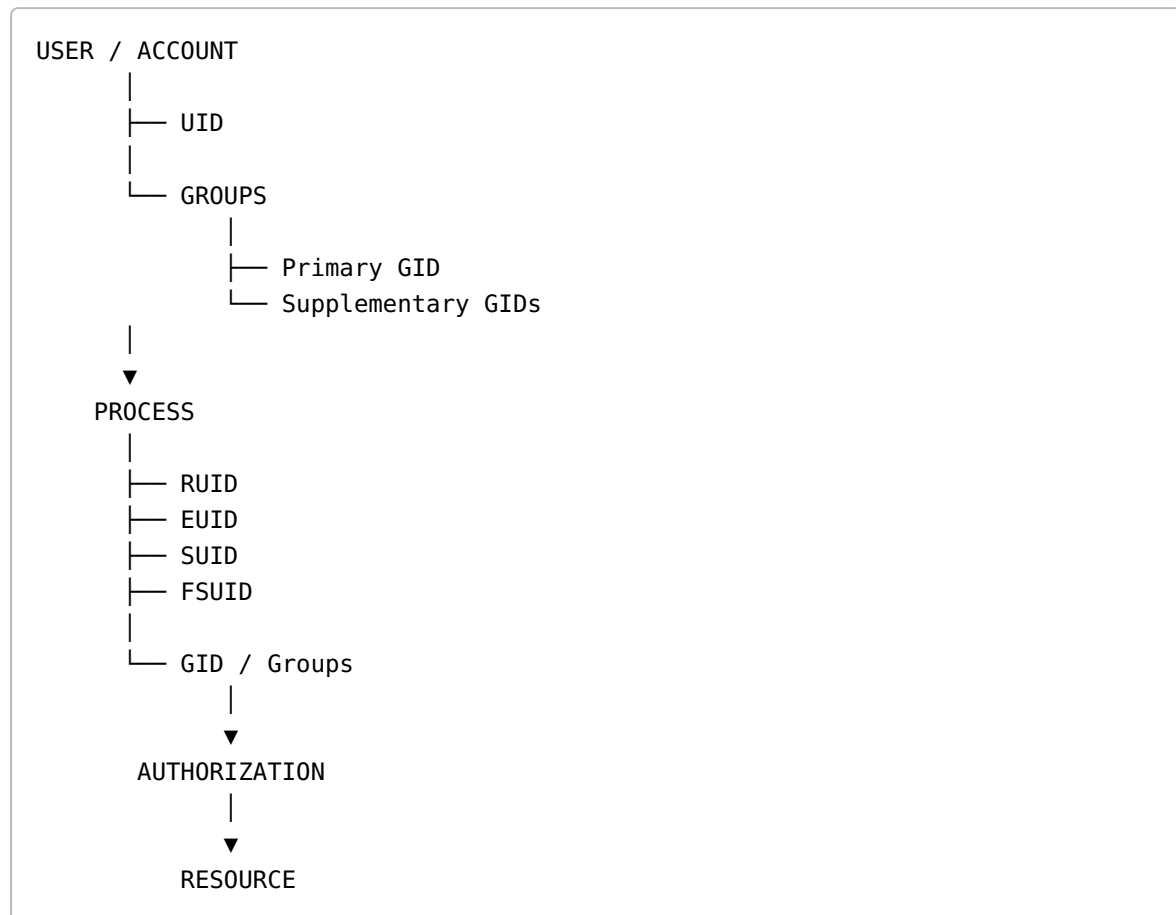
Kullanıcının ek grup üyelikleri.

PID

Process'in kimliği.

1.41 Ezberlenmesi Gereken Temel Model

Şu modeli zihninde tut:



Ve özellikle:

R = Real
E = Effective

S = Saved
FS = Filesystem

şeklinde hatırla.

En kritik ayırım ise:

RUID → "Hangi kullanıcı bağlamıyla ilişkili?"
EUID → "Hangi UID etkin?"
SUID → "Hangi UID saklanmış?"
FSUID → "Filesystem için hangi UID kullanılıyor?"

1.42 Bölüm Sonu — Bilmen Gerekenler

Bu bölümü tamamladıktan sonra aşağıdaki soruların cevabını verebilmelisin:

1. UID nedir?
2. UID ile identity arasındaki fark nedir?
3. UID 0 neden önemlidir?
4. System account neden vardır?
5. PID ile UID arasındaki fark nedir?
6. Process credential nedir?
7. RUID nedir?
8. EUID nedir?
9. RUID ve EUID neden farklı olabilir?
10. SUID nedir?
11. FSUID nedir?
12. Setuid bit ile Saved Set-user-ID arasındaki fark nedir?
13. GID nedir?
14. Primary group nedir?
15. Supplementary group nedir?
16. `/etc/passwd` ne işe yarar?
17. `/etc/group` ne işe yarar?
18. `id` komutu bize ne gösterir?
19. `/proc/<PID>/status` içindeki `Uid:` satırı ne anlatır?
20. Bir ITDR sistemi neden RUID ve EUID'yi ayrı izlemek isteyebilir?

Bu sorulara cevap verebiliyorsan Linux'un identity modelinin temelini anlamaya başlamışsın demektir.

Bölüm 1'in Ana Fikri

Linux açısından kimlik:

"kullanıcının adı"

değildir.

Daha doğru bir model:

```
Identity
  ↓
Account
  ↓
UID / GID / Groups
  ↓
Process Credentials
  ↓
Authorization
  ↓
Action
  ↓
Resource
```

şeklindedir.

Dolayısıyla ileride Linux security, privilege escalation, EDR ve ITDR çalışırken yalnızca:

"Bu işlemi Alice yaptı."

demek yerine:

```
Hangi identity?
Hangi account?
Hangi UID?
Hangi process?
Hangi RUID?
Hangi EUID?
Hangi group?
Hangi privilege?
Hangi action?
Hangi resource?
```

sorularını sormamız gerekir.

Bu, Linux'un identity modelinin güvenlik perspektifinden düşünülmeye başlandığı noktadır.